

AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança — Axis Kombat

Data: 24/09/2026

Escopo auditado: repositório 07Castiel/AxusKombat — código-fonte (TanStack Start (React 19) sobre Cloudflare Workers (Nitro)), migrações e RLS do Supabase/PostgreSQL, rotas públicas, worker de WhatsApp, integração Stripe, bundle do navegador e histórico git (400 commits).

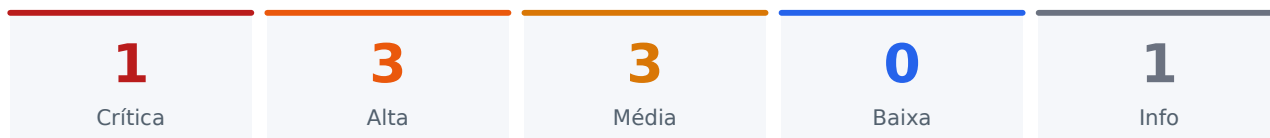
Total de achados: 8 (1 Crítica · 3 Alta · 3 Média · 1 Informativa).

Categorias: (1) isolamento de inquilino, (2) permissão no navegador, (3) IDOR, (4) chaves/segregos, (5) inputs sem tratamento (XSS).

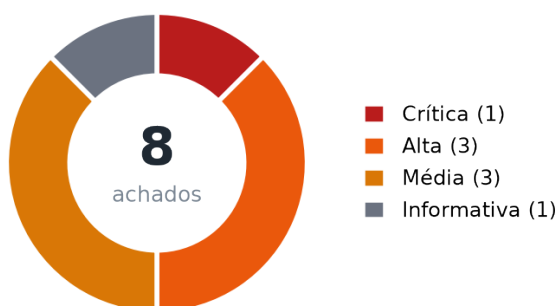
Nota metodológica. Cada categoria foi mapeada para o equivalente da stack detectada. O isolamento de inquilino do projeto é RLS no Postgres (`get_current_tenant`), não havendo Supabase 'sem RLS'; a verificação de permissão foi cruzada entre os gates do frontend (React) e o caminho de escrita real (server function ou escrita direta protegida por RLS); IDOR foi varrido nos 63 handlers e nas policies; segredos foram procurados na árvore, no bundle e em todo o histórico git; XSS foi avaliado no React (escape automático), nos sinks `dangerouslySetInnerHTML` e nas exportações. Os achados críticos foram reproduzidos em uma instância PostgreSQL 16 local montada a partir das migrações reais do repositório.

Resumo executivo

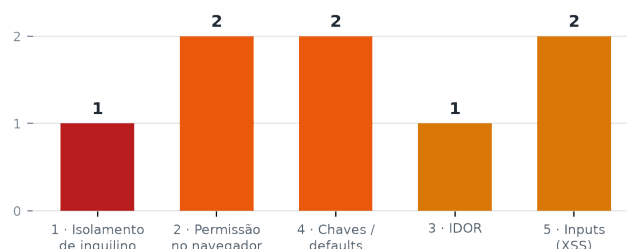
A auditoria identificou 8 achados, sendo 1 crítico e 3 de severidade alta. O risco central é a possibilidade de um usuário anônimo tomar conta dos dados de outra academia (quebra de isolamento multi-inquilino), seguido de duas escaladas de privilégio gravadas direto do navegador — burla de assinatura/suspensão e reset das próprias permissões. As fundações de segurança do projeto, porém, são sólidas: RLS em todas as tabelas, webhook do Stripe robusto, portal do aluno com 2º fator e nenhum segredo vivo no código ou no histórico git.



Achados por severidade



Achados por categoria



Cor da barra = maior severidade da categoria.

Panorama dos achados

#	Severidade	Categoria	Achado
1	CRÍTICA	1. Isolamento de inquilino	Tomada de conta de outra academia via perfil autoinserido (flag pública skip_tenant + policy profiles_insert_self sem trava de tenant)
2	ALTA	2. Permissão no navegador	Admin escreve o próprio status de assinatura e reativa academia suspensa direto do navegador (tenants_update_admin sem trava de coluna)
3	ALTA	2. Permissão no navegador	Funcionário reescreve as próprias permissões e anula as restrições do admin (profiles_update_self_or_admin cobre a coluna permissions)
4	ALTA	4. Chaves / defaults inseguros	Endpoints de cron aceitam a chave anon pública como segredo quando CRON_SECRET não está configurada, sem validação de startup
5	MÉDIA	3. IDOR	Contrato aceita aluno_id de outra academia (chave estrangeira cruzando tenant não é verificada em contratos_insert nem no handler)
6	MÉDIA	5. Inputs sem tratamento	Injeção de fórmula (CSV) na exportação de logs de acesso, com campos controlados por visitante anônimo, aberta pelo admin mestre
7	MÉDIA	4. Chaves / defaults inseguros	Defaults inseguros de segredo sem validação de startup: token do painel mestre assinado com a senha, e .env versionado no repositório
8	INFO	5. Inputs sem tratamento	dangerouslySetInnerHTML no componente de gráfico (CSS de tema)

Pontos fortes e pontos fracos

Pontos fortes (o que está protegido)

Itens verificados no código real e considerados corretos — comprovam a cobertura da auditoria.

- ✓ Isolamento por RLS em todas as tabelas de negócio. As 13 tabelas de operação têm RLS habilitada e policies escopadas por `get_current_tenant()`. alunos/mensalidades/contratos/etc. exigem papel na SELECT — o perfil autoinserido do achado nº 1 NÃO vaza a lista de alunos por RLS direta (só via server functions de módulo ausente).
- ✓ `has_role()` corrigido para exigir papel no tenant do perfil (C3). `user_roles.tenant_id` passou a ser conferido contra `profiles.tenant_id`. Verifiquei no PoC: o atacante que entra no tenant da vítima NÃO ganha papel admin ali — as funções `requireAdmin` permanecem barradas.
- ✓ `profiles`: mudança do próprio `tenant_id` por UPDATE bloqueada (C2). O WITH CHECK reaproveita `get_current_tenant()` (STABLE, snapshot antigo), então um perfil existente não consegue se mover de academia via UPDATE.
- ✓ Autenticação dos hooks de cron com comparação de tempo constante. Quando `CRON_SECRET` está setada, ela é o único segredo aceito e a chave anon é recusada; `secretsMatch` usa `timingSafeEqual` sem vaziar tamanho.
- ✓ Webhook do Stripe: assinatura verificada, idempotência e ordem. `constructEventAsync` valida a assinatura; `stripe_webhook_events` (PK em `event_id`) faz de-dupe e a guarda de ordem descarta eventos atrasados; o registro é liberado se o processamento falha, evitando perder o evento.
- ✓ Login do painel mestre endurecido. Teto de 5 tentativas/15min por IP (`master_login_attempts`), comparação de digests em tempo constante, token HMAC com TTL de 12h e auditoria em `system_logs`.
- ✓ Portal do aluno com 2º fator e sessões hashheadas. Acesso por matrícula + data de nascimento; sessão em cookie `httpOnly/secure/sameSite=strict` com token hashheadado (SHA-256); tetos por IP, por matrícula e diário; matrícula gerada por rejection sampling (sem vies).
- ✓ Endpoint público `track-visit` tratado. Corpo validado por zod, teto por IP, CORS restrito a `APP_URL/origem`, `user_id` obrigatoriamente UUID e geolocalização com cache — fecha o abuso anterior.
- ✓ Bloqueio de escrita por assinatura no próprio banco. O gatilho `tg_exigir_assinatura` em 12 tabelas recusa INSERT/UPDATE/DELETE de conta vencida mesmo se alguém chamar o Supabase direto — leitura permanece liberada (decisão consciente de LGPD).
- ✓ Logs da plataforma fora do alcance do cliente (C5) e bucket de fotos privado (M3). `visitor_logs` e `system_logs` tiveram SELECT/DELETE revogados de `authenticated`; o bucket fotos-alunos foi tornado privado e as policies escopadas por `tenant`.
- ✓ Sem segredos vivos no código, no bundle ou no histórico git. Varredura da árvore, do bundle compilado (só a chave anon pública aparece) e dos 400 commits: nenhuma `service_role`, `sk_live`, `whsec` ou chave privada. Os price IDs do Stripe no código não são segredos.
- ✓ Guardas server-side consistentes na maioria dos handlers. `requireAdmin/requirePermissao` resolvem o tenant sempre de `profiles` (fonte única), não de `user_roles`; handlers por id (`registrarPagamento`, `deleteDespesa`, `resendNotification`) usam o client RLS ou checam posse explicitamente (`resendNotification` compara `n.tenant_id`).

Pontos fracos (os riscos centrais)

#1 · Tomada de conta de outra academia via perfil autoinserido (flag pública `skip_tenant` + policy `profiles_insert_self` sem trava de `tenant`)

PoC reproduzida em Postgres 16 local com o schema real: o atacante passa a ler a linha `tenants` da vítima (CNPJ/CPF, chave PIX, e-mail e telefone do responsável, status) e a gravar arquivos na pasta de fotos da vítima. Pior: as server functions que usam `requirePermissao` (módulo ausente = liberado, pois perfis nascem com `permissions {}`) rodam com a `service_role` escopada ao `tenant` da vítima — `listStudentPortalAccess`, `issueAllStudentPortalAccess`, `frequenciaAluno`, `upsertContratoAtivo`, `upsertDespesa` e `togglePresenca` passam a ler e escrever os dados da vítima. A trava de papel (`has_role`, corrigida em C3) impede só as funções `requireAdmin`; o resto cai. É takeover multi-inquilino.

#2 · Admin escreve o próprio status de assinatura e reativa academia suspensa direto do navegador (tenants_update_admin sem trava de coluna)

PoC: um admin executa `update tenants set status='active', plan='elite', ativo=true, trial_ends_at=null where id=<meu tenant>` e passa a ter acesso pago sem pagar — `tenant_liberado()` retorna true. O mesmo UPDATE com `ativo=true` desfaz uma suspensão aplicada pelo admin mestre (moderação da plataforma anulada pelo próprio cliente). Também permite forjar `stripe_customer_id/subscription_id`, o que confunde o webhook.

#3 · Funcionário reescreve as próprias permissões e anula as restrições do admin (profiles_update_self_or_admin cobre a coluna permissions)

PoC: recepção com `permissions {alunos:{ver:false},pagamentos:{ver:false}}` executa `update profiles set permissions='{ }' where id=auth.uid()` e volta a ler alunos e a operar pagamentos. As restrições granulares (recurso vendido no plano Elite) viram decorativas — pior que não existir, dão falsa sensação de controle ao admin. Não vaza outro tenant (o papel segue escopado), mas escala privilégio dentro da academia.

#4 · Endpoints de cron aceitam a chave anon pública como segredo quando `CRON_SECRET` não está configurada, sem validação de startup

Se `CRON_SECRET` não estiver setada em produção, qualquer pessoa dispara o worker de WhatsApp em laço para TODAS as academias (o próprio comentário do código admite isso), processa/gera mensalidades e queima a reputação dos números de WhatsApp — negação de serviço e custo direto. É o padrão `${VAR:-default-público}` que vira segredo real por omissão.

Achados detalhados

Um bloco por achado, agrupado por categoria. Cada bloco traz severidade, arquivo:linha, trecho do código, por que é explorável, impacto, correção, critérios de aceite e condições de explorabilidade.

1. Isolamento de inquilino

CRÍTICA

#1 — Tomada de conta de outra academia via perfil autoinserido (flag pública skip_tenant + policy profiles_insert_self sem trava de tenant)

Arquivo: supabase/migrations/20260516170932_...sql:376 • supabase/HARDENING_7_APLICAR.sql:373 • src/routes/precos.tsx:467

Ref.: profiles_insert_self (WITH CHECK id = auth.uid()); handle_new_user IF skip_tenant THEN RETURN NEW; signUp(options.data)

```
CREATE POLICY "profiles_insert_self" ON public.profiles
FOR INSERT TO authenticated WITH CHECK (id = auth.uid());
-- handle_new_user():
IF COALESCE((NEW.raw_user_meta_data->>'skip_tenant')::boolean, false) THEN
RETURN NEW; -- não cria tenant/profile: a PK fica livre
-- precos.tsx (chamável direto com a chave anon):
supabase.auth.signUp({ options: { data: { skip_tenant: true } } })
```

Por que é explorável: O comentário do hardening deixa profiles_insert_self como está, raciocinando que 'o trigger já cria a linha no cadastro, então um INSERT do cliente sempre colide com a PK'. Isso deixa de valer quando o metadata traz skip_tenant:true — aí o trigger retorna sem criar nada e a chave primária de profiles (o id do próprio usuário) fica livre. Como a policy de INSERT só exige id = auth.uid() e NÃO amarra tenant_id, o atacante insere o próprio perfil apontando para o tenant_id de qualquer vítima. A partir daí get_current_tenant() devolve o tenant da vítima e todas as leituras escopadas só por tenant vazam.

Impacto: PoC reproduzida em Postgres 16 local com o schema real: o atacante passa a ler a linha tenants da vítima (CNPJ/CPF, chave PIX, e-mail e telefone do responsável, status) e a gravar arquivos na pasta de fotos da vítima. Pior: as server functions que usam requirePermissao (módulo ausente = liberado, pois perfis nascem com permissions {}) rodam com a service_role escopada ao tenant da vítima — listStudentPortalAccess, issueAllStudentPortalAccess, frequenciaAluno, upsertContratoAtivo, upsertDespesa e togglePresenca passam a ler e escrever os dados da vítima. A trava de papel (has_role, corrigida em C3) impede só as funções requireAdmin; o resto cai. É takeover multi-inquilino.

Explorabilidade: Alta. Basta a chave anon (pública, no bundle) e uma chamada signUp na API do Supabase. Não exige engenharia social nem privilégio prévio. Condição: a academia-alvo estar 'liberada' (o gatilho de assinatura barra INSERT em tenant vencido) e o atacante conhecer o tenant_id da vítima (que costuma aparecer em nomes de pasta de foto, e a própria falha ajuda a enumerar).

Correção sugerida: 1) Endurecer a policy de INSERT para exigir que o perfil nasça no próprio tenant e nunca num alheio — o caminho legítimo (trigger) roda como service_role e não é afetado. Ex.: WITH CHECK (id = auth.uid() AND tenant_id = public.get_current_tenant()) só resolve quando já existe perfil; como aqui não existe, o correto é bloquear o INSERT de profiles vindo de 'authenticated' por completo (só a service_role/trigger cria perfil) — DROP POLICY profiles_insert_self. 2) No trigger, ignorar skip_tenant vindo do metadata do usuário final: honrar o convite de equipe por um canal que o usuário não controla (createStaff já usa service_role), não por um campo do signUp público. 3) Defesa em profundidade: um UNIQUE/gatilho que impeça um profile cujo tenant não bata com nenhum papel do usuário.

Critérios de aceite:

- ☐ Um usuário autenticado NÃO consegue inserir/alterar profiles.tenant_id para um tenant onde não tem papel (teste automatizado em supabase/testes).
- ☐ signUp com {skip_tenant:true} no metadata não cria conta utilizável nem deixa a PK de profiles livre para inserção manual.
- ☐ get_current_tenant() de uma conta recém-criada por skip_tenant não resolve o tenant de outra academia.
- ☐ Teste de regressão cobrindo o cenário do PoC entra na suíte t45_tenant_perm.

2. Permissão no navegador

ALTA**#2 — Admin escreve o próprio status de assinatura e reativa academia suspensa direto do navegador (tenants_update_admin sem trava de coluna)**

Arquivo: supabase/migrations/20260516170932_...sql:370

Ref.: CREATE POLICY tenants_update_admin ... FOR UPDATE (sem WITH CHECK)

```
CREATE POLICY "tenants_update_admin" ON public.tenants
FOR UPDATE TO authenticated
USING (id = public.get_current_tenant() AND public.is_admin());
-- sem WITH CHECK e sem restrição de coluna: status/plan/ativo ficam abertos
```

Por que é explorável: A cobrança é decidida no servidor (webhook do Stripe grava tenants.status/plan) e a suspensão pelo painel mestre grava tenants.ativo. Mas a policy de UPDATE libera QUALQUER coluna da própria linha para o admin do tenant. O cliente escreve tenants direto pelo Supabase (padrão do projeto).

Impacto: PoC: um admin executa update tenants set status='active', plan='elite', ativo=true, trial_ends_at=null where id=<meu tenant> e passa a ter acesso pago sem pagar — tenant_liberado() retorna true. O mesmo UPDATE com ativo=true desfaz uma suspensão aplicada pelo admin mestre (moderação da plataforma anulada pelo próprio cliente). Também permite forjar stripe_customer_id/subscription_id, o que confunde o webhook.

Explorabilidade: Alta para quem é admin da própria academia (todo dono é admin). Exige só o console do navegador. Burla o paywall e a moderação da plataforma.

Correção sugerida: Tirar as colunas sensíveis do alcance do cliente. Opção A: restringir a policy a um subconjunto de colunas seguras via coluna-nível (GRANT UPDATE (nome, telefone, endereco, logo_url, pix_*, ...) e revogar UPDATE das demais) — RLS não filtra coluna, então a trava de coluna vem do GRANT. Opção B (recomendada): não deixar o cliente escrever tenants; toda alteração de cadastro passa por updateTenantConfig (server function, já existe) e as colunas de billing/ativo só são escritas por service_role (webhook/master).

Critérios de aceite:

- ☐ Admin autenticado NÃO consegue alterar status, plan, plan_period, is_trial, trial_ends_at, ativo, stripe_customer_id, stripe_subscription_id.
- ☐ Alterar nome/telefone/endereço/pix continua funcionando pela tela.
- ☐ Teste: UPDATE do cliente nas colunas de billing/ativo é recusado pelo banco.

ALTA**#3 — Funcionário reescreve as próprias permissões e anula as restrições do admin (profiles_update_self_or_admin cobre a coluna permissions)**

Arquivo: supabase/migrations/20260901000000_hardening_multitenant.sql:52 • src/lib/permissions.ts:13

Ref.: profiles_update_self_or_admin (WITH CHECK permite id = auth.uid())

```
CREATE POLICY profiles_update_self_or_admin ON public.profiles
FOR UPDATE ... USING (id = auth.uid() OR (... is_admin()))
WITH CHECK (tenant_id = get_current_tenant() AND (id=auth.uid() OR is_admin()));
-- nada protege a coluna permissions: o próprio usuário a reescreve
```

Por que é explorável: A camada A7 de permissões por módulo é gravada em profiles.permissions e o invariante declarado é 'permissão só RESTRINGE'. Mas a policy de UPDATE do perfil deixa o próprio usuário escrever qualquer coluna da própria linha, inclusive permissions. Um recepcionista/financeiro/professor a quem o admin tirou 'editar pagamentos' faz update profiles set permissions='{}' e recupera tudo — via console do navegador, sem passar por server function.

Impacto: PoC: recepção com permissions {alunos:{ver:false},pagamentos:{ver:false}} executa update profiles set permissions='{}' where id=auth.uid() e volta a ler alunos e a operar pagamentos. As restrições granulares (recurso vendido no plano Elite) viram decorativas — pior que não existir, dão falsa sensação de controle ao admin. Não vaza outro tenant (o papel segue escopado), mas escala privilégio dentro da academia.

Explorabilidade: Alta para qualquer funcionário com login. Só o console do navegador.

Correção sugerida: Impedir que o usuário altere a própria coluna permissions (e papéis). Como RLS não filtra coluna: revogar UPDATE(permissions) de authenticated e deixar a escrita só por createStaff/updateStaff (service_role, já exigem requireAdmin). Alternativa: gatilho BEFORE UPDATE que rejeite mudança de permissions quando auth.uid() = NEW.id e não is_admin().

Critérios de aceite:

- ☐ Usuário não-admin NÃO consegue alterar a própria coluna permissions nem inserir/alterar user_roles.
- ☐ Admin continua ajustando permissões da equipe pela tela Equipe.
- ☐ Teste automatizado do auto-UPDATE de permissions recusado pelo banco.

4. Chaves / defaults inseguros

ALTA

#4 — Endpoints de cron aceitam a chave anon pública como segredo quando CRON_SECRET não está configurada, sem validação de startup

Arquivo: src/lib/cron-auth.ts:47 • src/lib/cron-auth.ts:70

Ref.: authorizeCronRequest (fallback legado) e internalCronSecret()

```
if (cronSecret) { ...compara CRON_SECRET... }
// Modo legado – ainda vulnerável:
const legacy = process.env.SUPABASE_PUBLISHABLE_KEY;
if (!legacy || !secretsMatch(received, legacy)) return unauthorized();
return { ok: true }; // aceita a chave anon, que é pública
```

Por que é explorável: Quando CRON_SECRET não está definida, os hooks públicos (/api/public/hooks/dispatch-notifications e notify-mensalidades) voltam a aceitar o header com SUPABASE_PUBLISHABLE_KEY — o mesmo JWT role=anon que o Vite injeta no bundle e qualquer visitante lê no DevTools. O código só 'grita no log'; não há validação de startup que recuse subir nesse estado.

Impacto: Se CRON_SECRET não estiver setada em produção, qualquer pessoa dispara o worker de WhatsApp em laço para TODAS as academias (o próprio comentário do código admite isso), processa/gera mensalidades e queima a reputação dos números de WhatsApp — negação de serviço e custo direto. É o padrão \${VAR:-default-público} que vira segredo real por omissão.

Explorabilidade: Alta SE CRON_SECRET não estiver configurada (condição de deploy). A chave anon necessária é pública. O .env.example marca CRON_SECRET como 'troque-por-um-valor...', o que sugere que ambientes podem subir sem defini-la.

Correção sugerida: 1) Exigir CRON_SECRET na inicialização: se ausente, recusar as rotas de hook (retornar 503) em vez de cair no fallback anon. 2) Remover o fallback para SUPABASE_PUBLISHABLE_KEY. 3)

Documentar CRON_SECRET como obrigatória no deploy e conferir em notification_worker_runs.

Critérios de aceite:

- ☐ Sem CRON_SECRET, os hooks respondem 503/401 — nunca aceitam a chave anon.
- ☐ Com CRON_SECRET setada, chamada sem o header correto é 401.
- ☐ Startup (ou o primeiro hit) valida a presença de CRON_SECRET e registra.

3. IDOR

MÉDIA

#5 — Contrato aceita aluno_id de outra academia (chave estrangeira cruzando tenant não é verificada em contratos_insert nem no handler)

Arquivo: supabase/migrations/20260701033153_...sql:268 • src/lib/contratos.functions.ts:84

Ref.: contratos_insert WITH CHECK (só confere tenant_id do contrato, não do aluno); upsertContratoAtivo insere aluno_id sem checar posse

```
CREATE POLICY contratos_insert ... WITH CHECK (
tenant_id = get_current_tenant() AND (is_admin() OR is_recepcao() ...));
-- aluno_id não é conferido contra o tenant do chamador
// contratos.functions.ts insere { tenant_id: tenantId, aluno_id: data.aluno_id }
```

Por que é explorável: A policy garante que o contrato nasce no tenant de quem chama, mas não exige que o aluno_id referenciado também seja daquele tenant. upsertContratoAtivo insere o aluno_id da entrada sem um select de posse. Mesmo padrão em historico_graduacoes e no antigo matriculas.

Impacto: PoC: admin da Academia A cria contrato no próprio tenant apontando para um aluno da Academia B; gerar_mensalidades_contrato (service_role) gera as mensalidades e o worker, cujo join notificacoes→alunos!inner roda como service_role, lê nome e telefone do aluno da vítima e enviaria WhatsApp em nome da academia errada. Isoladamente exige conhecer o UUID do aluno-alvo (v4 aleatório); combinado com o achado nº 1, que entrega esses UUIDs, vira alto.

Explorabilidade: Média isolada (precisa do UUID do aluno da vítima). Alta em conjunto com o nº 1.

Correção sugerida: No upsertContratoAtivo (e em quem grava aluno_id/historico), validar que o aluno pertence ao tenant antes de inserir (select id from alunos where id=aluno_id and tenant_id=tenantId).

Reforçar no banco com CHECK/gatilho que exija alunos.tenant_id = contratos.tenant_id.

Critérios de aceite:

- ☐ Criar contrato/histórico com aluno_id de outro tenant é recusado (server e banco).
- ☐ Fluxo normal (aluno do próprio tenant) segue funcionando.
- ☐ Teste de regressão do cenário cross-tenant na suíte.

5. Inputs sem tratamento

MÉDIA

#6 — Injeção de fórmula (CSV) na exportação de logs de acesso, com campos controlados por visitante anônimo, aberta pelo admin mestre

Arquivo: src/routes/admin-master.acessos.tsx:47 • src/lib/acessos.functions.ts:133 •

src/routes/api/public/track-visit.ts:153

Ref.: toCsv() escapa aspas/vírgula mas não neutraliza = + - @ iniciais

```
const escape = (v) => { const s = String(v).replace(/"/g, '""');
return /[",\n;]/.test(s) ? `"${s}"` : s; }; // não trata = + - @
// current_page, referrer, user_agent vêm de track-visit (visitante anônimo)
```

Por que é explorável: visitor_logs guarda current_page, referrer, user_agent e browser vindos do endpoint público track-visit, ou seja, controlados por qualquer visitante. A exportação toCsv() só cuida de aspas/; e não prefixa as células que começam com = + - @. Ao abrir o CSV no Excel/Sheets, o admin mestre executa a fórmula.

Impacto: Um visitante anônimo grava um referrer como =HYPERLINK(...) ou fórmula de exfiltração/comando; quando o admin da plataforma exporta e abre a planilha, a fórmula roda na máquina dele (injeção de fórmula / possível execução via recursos do Excel). O mesmo padrão, com risco menor, na exportação XLSX de alunos (nomes internos).

Explorabilidade: Média. Entrada é anônima e trivial (headers HTTP); o gatilho é o admin abrir o CSV. Impacto recai sobre a conta mais privilegiada (painel mestre).

Correção sugerida: Prefixar com apóstrofo (') toda célula que comece com = + - @ | tab, ou encapsular como texto, tanto no toCsv de acessos quanto no CSV de relatórios e no XLSX de alunos. Idealmente uma função de sanitização compartilhada.

Critérios de aceite:

- ☐ Célula iniciada por = + - @ é neutralizada em toda exportação (CSV e XLSX).
- ☐ Valores legítimos (números, texto normal) permanecem intactos.
- ☐ Teste unitário de toCsv com payload de fórmula.

4. Chaves / defaults inseguros

MÉDIA**#7 — Defaults inseguros de segredo sem validação de startup: token do painel mestre assinado com a senha, e .env versionado no repositório**

Arquivo: src/lib/master-token.server.ts:33 • .env • .gitignore

Ref.: getSecret() cai em MASTER_ADMIN_PASSWORD; .env rastreado no git

```
function getSecret() {
const dedicada = process.env.MASTER_TOKEN_SECRET;
if (dedicada) return dedicada;
const s = process.env.MASTER_ADMIN_PASSWORD; // fallback: senha vira chave
...console.warn(...); return s; }
# .env está no versionamento (contém hoje só a chave anon pública)
```

Por que é explorável: Dois defaults que só avisam no log, sem trava. (a) Sem MASTER_TOKEN_SECRET, o token de sessão do /admin-master é assinado com a própria MASTER_ADMIN_PASSWORD — quem obtiver um token pode atacá-lo offline para recuperar a senha do super-admin. (b) O .env está versionado (o .gitignore não o cobre); hoje só carrega a chave anon (pública), mas o padrão convida um segredo real a vaziar no próximo commit — o próprio .env.example alerta 'NUNCA acrescente segredo'.

Impacto: (a) Recuperação offline da senha do painel que enxerga TODAS as academias, se o segredo dedicado não for configurado. (b) Superfície latente: qualquer

SUPABASE_SERVICE_ROLE_KEY/STRIPE_SECRET_KEY colocada no .env por engano entra no histórico git imediatamente. Verifiquei: hoje o histórico só tem a chave anon; nenhum segredo vivo foi commitado.

Explorabilidade: Média e condicional à ausência dos segredos dedicados. Sem exposição ativa hoje, mas remove uma rede de segurança inteira quando a config falha.

Correção sugerida: 1) Exigir MASTER_TOKEN_SECRET (distinto da senha) na inicialização; recusar login mestre se ausente, em vez de assinar com a senha. 2) Remover .env do versionamento (git rm --cached .env) e adicioná-lo ao .gitignore; manter só o .env.example. 3) Validação de startup que rejeite defaults conhecidos ('troque-por-um-valor...').

Critérios de aceite:

- ☐ Sem MASTER_TOKEN_SECRET dedicado, o login mestre é recusado (não usa a senha).
- ☐ .env deixa de ser rastreado; .gitignore o cobre; .env.example permanece.
- ☐ Validação de startup recusa valores default de placeholder.

5. Inputs sem tratamento

INFO**#8 — dangerouslySetInnerHTML no componente de gráfico (CSS de tema)**

Arquivo: src/components/ui/chart.tsx:73

Ref.: ChartStyle injeta <style> a partir do config de cores

```
<style dangerouslySetInnerHTML={{ __html: ...color config... }} />
```

Por que é explorável: É o componente Chart do shadcn/ui. O __html é montado a partir das cores do objeto config, que nas telas do projeto são definidas pelo desenvolvedor (constantes), não por entrada de usuário. Não encontrei caminho em que dado de usuário chegue ao config de cor.

Impacto: Sem impacto explorável hoje: os valores são estáticos. Fica registrado como ponto de atenção — se algum dia uma cor de gráfico passar a vir do banco/usuário, isso vira vetor de injeção de CSS.

Explorabilidade: Nenhuma hoje (entrada é de desenvolvedor). Informativa/preventiva.

Correção sugerida: Manter os valores de cor restritos a um enum/paleta fixa; nunca interpolar string vinda de usuário no config do gráfico. Opcional: validar o formato hex antes de injetar.

Critérios de aceite:

- ☐ Cores de gráfico permanecem constantes de desenvolvedor.
- ☐ Se algum dia forem dinâmicas, passam por validação de formato hex.

Recomendações priorizadas

P1 = corrigir antes de qualquer novo release. P2 = próximo ciclo. P3 = higiene e prevenção.

P1	Fechar a tomada de conta de tenant (achado nº 1): bloquear INSERT de profiles por 'authenticated' e ignorar skip_tenant vindo do signUp público. É o único achado que dá leitura/escrita cruzada entre academias.
P1	Tirar billing/ativo e permissions do alcance de escrita do cliente (achados 2 e 3): via GRANT de coluna ou movendo a escrita para server functions/service_role. Fecha burla de paywall, anulação de suspensão e escalonamento de privilégio interno.
P1	Tornar CRON_SECRET e MASTER_TOKEN_SECRET obrigatórios com validação de startup (achados 4 e 7a); remover os fallbacks para a chave anon e para a senha do mestre.
P2	Verificar posse de aluno_id ao gravar contrato/histórico (achado nº 5), no handler e com CHECK no banco.
P2	Neutralizar injeção de fórmula em todas as exportações CSV/XLSX (achado nº 6) com uma função de sanitização compartilhada.
P3	Remover .env do versionamento e cobri-lo no .gitignore (achado nº 7b); manter apenas .env.example.
P3	Manter as cores de gráfico restritas a paleta fixa (achado nº 8) e adicionar testes de regressão de RLS cruzada na suíte supabase/testes.

Issues para o GitHub

Texto completo de cada issue em Markdown, pronto para copiar e colar. Cada uma está entre os delimitadores --- ISSUE n --- e --- FIM ISSUE n ---. Achados triviais do mesmo tema foram agrupados para não gerar spam de issues.

--- ISSUE 1 ---

****Título:**** [Segurança] Tomada de conta de outra academia via perfil autoinserido

****Labels:**** `security`, `critical`

****Severidade:**** Crítica

Problema e por que é explorável

O comentário do hardening deixa `profiles_insert_self` como está, raciocinando que 'o trigger já cria a linha no cadastro, então um INSERT do cliente sempre colide com a PK'. Isso deixa de valer quando o metadata traz `skip_tenant:true` – aí o trigger retorna sem criar nada e a chave primária de `profiles` (o id do próprio usuário) fica livre. Como a policy de INSERT só exige `id = auth.uid()` e NÃO amarra `tenant_id`, o atacante insere o próprio perfil apontando para o `tenant_id` de qualquer vítima. A partir daí `get_current_tenant()` devolve o tenant da vítima e todas as leituras escopadas só por tenant vazam.

Evidência

`supabase/migrations/20260516170932\_...sql:376 • supabase/HARDENING\_7\_APLICAR.sql:373 • src/routes/precos.tsx:467`

...

```
CREATE POLICY "profiles_insert_self" ON public.profiles
FOR INSERT TO authenticated WITH CHECK (id = auth.uid());
-- handle_new_user():
IF COALESCE((NEW.raw_user_meta_data->'skip_tenant')::boolean,false) THEN
RETURN NEW; -- não cria tenant/profile: a PK fica livre
-- precos.tsx (chamável direto com a chave anon):
supabase.auth.signUp({ options:{ data:{ skip_tenant:true } }})
...
```

Impacto

PoC reproduzida em Postgres 16 local com o schema real: o atacante passa a ler a linha tenants da vítima (CNPJ/CPF, chave PIX, e-mail e telefone do responsável, status) e a gravar arquivos na pasta de fotos da vítima. Pior: as server functions que usam `requirePermissao` (módulo ausente = liberado, pois perfis nascem com `permissions {}`) rodam com a `service_role` escopada ao tenant da vítima – `listStudentPortalAccess`, `issueAllStudentPortalAccess`, `frequenciaAluno`, `upsertContratoAtivo`, `upsertDespesa` e `togglePresenca` passam a ler e escrever os dados da vítima. A trava de papel (`has_role`, corrigida em C3) impede só as funções `requireAdmin`; o resto cai. É takeover multi-inquilino.

Explorabilidade

Alta. Basta a chave anon (pública, no bundle) e uma chamada `signUp` na API do Supabase. Não exige engenharia social nem privilégio prévio. Condição: a academia-alvo estar 'liberada' (o gatilho de assinatura barra INSERT em tenant vencido) e o atacante conhecer o `tenant_id` da vítima (que costuma aparecer em nomes de pasta de foto, e a própria falha ajuda a enumerar).

Sugestão de correção

1) Endurecer a policy de INSERT para exigir que o perfil nasça no próprio tenant e nunca num alheio – o caminho legítimo (trigger) roda como `service_role` e não é afetado. Ex.: `WITH CHECK (id = auth.uid() AND tenant_id = public.get_current_tenant())` só resolve quando já existe perfil; como aqui não existe, o correto é bloquear o INSERT de `profiles` vindo de 'authenticated' por completo (só a `service_role/trigger` cria perfil) – DROP POLICY `profiles_insert_self`. 2) No trigger, ignorar `skip_tenant` vindo do metadata do usuário final: honrar o convite de equipe por um canal que o usuário não controla (`createStaff` já usa `service_role`), não por um campo do `signUp` público. 3) Defesa em profundidade: um UNIQUE/gatilho que impeça um profile cujo tenant não bata com nenhum papel do usuário.

Critérios de aceite

- [] Um usuário autenticado NÃO consegue inserir/alterar `profiles.tenant_id` para um tenant onde não tem papel (teste automatizado em `supabase/testes`).
- [] `signUp` com `{skip_tenant:true}` no metadata não cria conta utilizável nem deixa a PK de `profiles` livre para inserção manual.
- [] `get_current_tenant()` de uma conta recém-criada por `skip_tenant` não resolve o tenant de outra academia.
- [] Teste de regressão cobrindo o cenário do PoC entra na suíte `t45_tenant_perm`.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

****Título:**** [Segurança] Admin escreve o próprio status de assinatura e reativa academia suspensa dire...

****Labels:**** `security`, `high`

****Severidade:**** Alta

Problema e por que é explorável

A cobrança é decidida no servidor (webhook do Stripe grava tenants.status/plan) e a suspensão pelo painel mestre grava tenants.ativo. Mas a policy de UPDATE libera QUALQUER coluna da própria linha para o admin do tenant. O cliente escreve tenants direto pelo Supabase (padrão do projeto).

Evidência

`supabase/migrations/20260516170932\_...sql:370`

```

```
CREATE POLICY "tenants_update_admin" ON public.tenants
FOR UPDATE TO authenticated
USING (id = public.get_current_tenant() AND public.is_admin());
-- sem WITH CHECK e sem restrição de coluna: status/plan/ativo ficam abertos
```
```

Impacto

PoC: um admin executa update tenants set status='active', plan='elite', ativo=true, trial_ends_at=null where id=<meu tenant> e passa a ter acesso pago sem pagar – tenant_liberado() retorna true. O mesmo UPDATE com ativo=true desfaz uma suspensão aplicada pelo admin mestre (moderação da plataforma anulada pelo próprio cliente). Também permite forjar stripe_customer_id/subscription_id, o que confunde o webhook.

Explorabilidade

Alta para quem é admin da própria academia (todo dono é admin). Exige só o console do navegador. Burla o paywall e a moderação da plataforma.

Sugestão de correção

Tirar as colunas sensíveis do alcance do cliente. Opção A: restringir a policy a um subconjunto de colunas seguras via coluna-nível (GRANT UPDATE (nome, telefone, endereco, logo_url, pix_*, ...) e revogar UPDATE das demais) – RLS não filtra coluna, então a trava de coluna vem do GRANT. Opção B (recomendada): não deixar o cliente escrever tenants; toda alteração de cadastro passa por updateTenantConfig (server function, já existe) e as colunas de billing/ativo só são escritas por service_role (webhook/master).

Critérios de aceite

- [] Admin autenticado NÃO consegue alterar status, plan, plan_period, is_trial, trial_ends_at, ativo, stripe_customer_id, stripe_subscription_id.
- [] Alterar nome/telefone/endereço/pix continua funcionando pela tela.
- [] Teste: UPDATE do cliente nas colunas de billing/ativo é recusado pelo banco.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

****Título:**** [Segurança] Funcionário reescreve as próprias permissões e anula as restrições do admin
****Labels:**** `security`, `high`

****Severidade:**** Alta

Problema e por que é explorável

A camada A7 de permissões por módulo é gravada em `profiles.permissions` e o invariante declarado é 'permissão só RESTRINGE'. Mas a policy de UPDATE do perfil deixa o próprio usuário escrever qualquer coluna da própria linha, inclusive `permissions`. Um recepcionista/financeiro/professor a quem o admin tirou 'editar pagamentos' faz `update profiles set permissions='{}'` e recupera tudo – via console do navegador, sem passar por `server function`.

Evidência

`supabase/migrations/20260901000000\_hardenning\_multitenant.sql:52 • src/lib/permissoes.ts:13`

```

```
CREATE POLICY profiles_update_self_or_admin ON public.profiles
FOR UPDATE ... USING (id = auth.uid() OR (... is_admin()))
WITH CHECK (tenant_id = get_current_tenant() AND (id=auth.uid() OR is_admin()));
-- nada protege a coluna permissions: o próprio usuário a reescreve
```
```

Impacto

PoC: recepção com `permissions {alunos:{ver:false},pagamentos:{ver:false}}` executa `update profiles set permissions='{}'` where `id=auth.uid()` e volta a ler alunos e a operar pagamentos. As restrições granulares (recurso vendido no plano Elite) viram decorativas – pior que não existir, dão falsa sensação de controle ao admin. Não vaza outro tenant (o papel segue escopado), mas escala privilégio dentro da academia.

Explorabilidade

Alta para qualquer funcionário com login. Só o console do navegador.

Sugestão de correção

Impedir que o usuário altere a própria coluna `permissions` (e papéis). Como RLS não filtra coluna: revogar `UPDATE(permissions)` de `authenticated` e deixar a escrita só por `createStaff/updateStaff` (`service_role`, já exigem `requireAdmin`). Alternativa: gatilho `BEFORE UPDATE` que rejeite mudança de `permissions` quando `auth.uid() = NEW.id` e não `is_admin()`.

Critérios de aceite

- [] Usuário não-admin NÃO consegue alterar a própria coluna `permissions` nem inserir/alterar `user_roles`.
- [] Admin continua ajustando permissões da equipe pela tela Equipe.
- [] Teste automatizado do auto-UPDATE de `permissions` recusado pelo banco.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

****Título:**** [Segurança] Endpoints de cron aceitam a chave anon pública como segredo quando CRON_SECRET...

****Labels:**** `security`, `high`

****Severidade:**** Alta

Problema e por que é explorável

Quando CRON_SECRET não está definida, os hooks públicos (/api/public/hooks/dispatch-notifications e notify-mensalidades) voltam a aceitar o header com SUPABASE_PUBLISHABLE_KEY – o mesmo JWT role=anon que o Vite injeta no bundle e qualquer visitante lê no DevTools. O código só 'grita no log'; não há validação de startup que recuse subir nesse estado.

Evidência

`src/lib/cron-auth.ts:47 • src/lib/cron-auth.ts:70`

```

```
if (cronSecret) { ...compara CRON_SECRET... }
// Modo legado – ainda vulnerável:
const legacy = process.env.SUPABASE_PUBLISHABLE_KEY;
if (!legacy || !secretsMatch(received, legacy)) return unauthorized();
return { ok: true }; // aceita a chave anon, que é pública
```
```

Impacto

Se CRON_SECRET não estiver setada em produção, qualquer pessoa dispara o worker de WhatsApp em laço para TODAS as academias (o próprio comentário do código admite isso), processa/gera mensalidades e queima a reputação dos números de WhatsApp – negação de serviço e custo direto. É o padrão \${VAR:-default-público} que vira segredo real por omissão.

Explorabilidade

Alta SE CRON_SECRET não estiver configurada (condição de deploy). A chave anon necessária é pública. O .env.example marca CRON_SECRET como 'troque-por-um-valor...', o que sugere que ambientes podem subir sem defini-la.

Sugestão de correção

1) Exigir CRON_SECRET na inicialização: se ausente, recusar as rotas de hook (retornar 503) em vez de cair no fallback anon. 2) Remover o fallback para SUPABASE_PUBLISHABLE_KEY. 3) Documentar CRON_SECRET como obrigatória no deploy e conferir em notification_worker_runs.

Critérios de aceite

- [] Sem CRON_SECRET, os hooks respondem 503/401 – nunca aceitam a chave anon.
- [] Com CRON_SECRET setada, chamada sem o header correto é 401.
- [] Startup (ou o primeiro hit) valida a presença de CRON_SECRET e registra.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

****Título:**** [Segurança] Contrato aceita aluno_id de outra academia

****Labels:**** `security`, `medium`

****Severidade:**** Média

Problema e por que é explorável

A policy garante que o contrato nasce no tenant de quem chama, mas não exige que o aluno_id referenciado também seja daquele tenant. upsertContratoAtivo insere o aluno_id da entrada sem um select de posse. Mesmo padrão em historico_graduacoes e no antigo matriculas.

Evidência

`supabase/migrations/20260701033153\_...sql:268 • src/lib/contratos.functions.ts:84`

```

```
CREATE POLICY contratos_insert ... WITH CHECK (
tenant_id = get_current_tenant() AND (is_admin() OR is_recepcao() ...));
-- aluno_id não é conferido contra o tenant do chamador
// contratos.functions.ts insere { tenant_id: tenantId, aluno_id: data.aluno_id }
```
```

Impacto

PoC: admin da Academia A cria contrato no próprio tenant apontando para um aluno da Academia B; gerar_mensalidades_contrato (service_role) gera as mensalidades e o worker, cujo join notificacoes→alunos!inner roda como service_role, lê nome e telefone do aluno da vítima e enviaria WhatsApp em nome da academia errada. Isoladamente exige conhecer o UUID do aluno-alvo (v4 aleatório); combinado com o achado nº 1, que entrega esses UUIDs, vira alto.

Explorabilidade

Média isolada (precisa do UUID do aluno da vítima). Alta em conjunto com o nº 1.

Sugestão de correção

No upsertContratoAtivo (e em quem grava aluno_id/historico), validar que o aluno pertence ao tenant antes de inserir (select id from alunos where id=aluno_id and tenant_id=tenantId). Reforçar no banco com CHECK/gatilho que exija alunos.tenant_id = contratos.tenant_id.

Critérios de aceite

- [] Criar contrato/histórico com aluno_id de outro tenant é recusado (server e banco).
- [] Fluxo normal (aluno do próprio tenant) segue funcionando.
- [] Teste de regressão do cenário cross-tenant na suíte.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

****Título:**** [Segurança] Injeção de fórmula

****Labels:**** `security`, `medium`

****Severidade:**** Média

Problema e por que é explorável

visitor_logs guarda current_page, referrer, user_agent e browser vindos do endpoint público track-visit, ou seja, controlados por qualquer visitante. A exportação toCsv() só cuida de aspas/; e não prefixa as células que começam com = + - @. Ao abrir o CSV no Excel/Sheets, o admin mestre executa a fórmula.

Evidência

`src/routes/admin-master.acessos.tsx:47 • src/lib/acessos.functions.ts:133 • src/routes/api/public/track-visit.ts:153`

```

```
const escape = (v) => { const s = String(v).replace(/"/g, '"');
return /[",\n;]/.test(s) ? `"${s}"` : s; }; // não trata = + - @
// current_page, referrer, user_agent vêm de track-visit (visitante anônimo)
```
```

Impacto

Um visitante anônimo grava um referrer como =HYPERLINK(...) ou fórmula de exfiltração/comando; quando o admin da plataforma exporta e abre a planilha, a fórmula roda na máquina dele (injeção de fórmula / possível execução via recursos do Excel). O mesmo padrão, com risco menor, na exportação XLSX de alunos (nomes internos).

Explorabilidade

Média. Entrada é anônima e trivial (headers HTTP); o gatilho é o admin abrir o CSV. Impacto recai sobre a conta mais privilegiada (painel mestre).

Sugestão de correção

Prefixar com apóstrofo (') toda célula que comece com = + - @ | tab, ou encapsular como texto, tanto no toCsv de acessos quanto no CSV de relatórios e no XLSX de alunos. Idealmente uma função de sanitização compartilhada.

Critérios de aceite

- [] Célula iniciada por = + - @ é neutralizada em toda exportação (CSV e XLSX).
- [] Valores legítimos (números, texto normal) permanecem intactos.
- [] Teste unitário de toCsv com payload de fórmula.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

****Título:**** [Segurança] Defaults inseguros de segredo sem validação de startup: token do painel maestr...

****Labels:**** `security`, `medium`

****Severidade:**** Média

Problema e por que é explorável

Dois defaults que só avisam no log, sem trava. (a) Sem MASTER_TOKEN_SECRET, o token de sessão do /admin-master é assinado com a própria MASTER_ADMIN_PASSWORD – quem obtiver um token pode atacá-lo offline para recuperar a senha do super-admin. (b) O .env está versionado (o .gitignore não o cobre); hoje só carrega a chave anon (pública), mas o padrão convida um segredo real a vaziar no próximo commit – o próprio .env.example alerta 'NUNCA acrescente segredo'.

Evidência

`src/lib/master-token.server.ts:33 • .env • .gitignore`

...

```
function getSecret() {
const dedicada = process.env.MASTER_TOKEN_SECRET;
if (dedicada) return dedicada;
const s = process.env.MASTER_ADMIN_PASSWORD; // fallback: senha vira chave
...console.warn(...); return s; }
# .env está no versionamento (contém hoje só a chave anon pública)
...
```

Impacto

(a) Recuperação offline da senha do painel que enxerga TODAS as academias, se o segredo dedicado não for configurado. (b) Superfície latente: qualquer SUPABASE_SERVICE_ROLE_KEY/STRIPE_SECRET_KEY colocada no .env por engano entra no histórico git imediatamente. Verifiquei: hoje o histórico só tem a chave anon; nenhum segredo vivo foi commitado.

Explorabilidade

Média e condicional à ausência dos segredos dedicados. Sem exposição ativa hoje, mas remove uma rede de segurança inteira quando a config falha.

Sugestão de correção

1) Exigir MASTER_TOKEN_SECRET (distinto da senha) na inicialização; recusar login mestre se ausente, em vez de assinar com a senha. 2) Remover .env do versionamento (git rm --cached .env) e adicioná-lo ao .gitignore; manter só o .env.example. 3) Validação de startup que rejeite defaults conhecidos ('troque-por-um-valor...').

Critérios de aceite

- [] Sem MASTER_TOKEN_SECRET dedicado, o login mestre é recusado (não usa a senha).
- [] .env deixa de ser rastreado; .gitignore o cobre; .env.example permanece.
- [] Validação de startup recusa valores default de placeholder.

--- FIM ISSUE 7 ---